

C Programming - Assignment 10

Q.1 Read a File Character by Character and Count Uppercase, Lowercase, Digits and Special Characters

Problem Statement

Write a program to read a file character by character. Count the number of uppercase letters, lowercase letters, digits, and special characters present in the file, and display the results on the console.

Concepts Used

- File Handling
- `FILE` Pointer
- `fopen()`
- `fgetc()`
- `fclose()`
- Looping (`while`)
- Conditional Statements (`if`)
- Character Classification (`isupper()`, `islower()`, `isdigit()`)

Step-by-Step Approach

1. Open the file in read mode using `fopen()`.
2. Check whether the file is opened successfully.
3. Read one character at a time using `fgetc()`.
4. Check whether the character is uppercase, lowercase, digit, or special character.
5. Increment the corresponding counter.
6. Continue until the end of the file (`EOF`).
7. Display the total counts.
8. Close the file using `fclose()`.

Test Cases

Test Case 1

File Content

```
Sunbeam123@#
```

Output

```
Uppercase Letters : 1  
Lowercase Letters : 6
```

```
Digits          : 3  
Special Characters: 2
```

Test Case 2

File Content

```
Hello@2025
```

Output

```
Uppercase Letters : 1  
Lowercase Letters : 4  
Digits            : 4  
Special Characters: 1
```

Q.2 Allocate Memory Dynamically for a 2D Array

Problem Statement

Write a program to allocate memory dynamically for a two-dimensional array, accept elements from the user, and display the array.

Concepts Used

- Dynamic Memory Allocation
- Double Pointer (**)
- `malloc()`
- `free()`
- Nested Loops (`for`)
- Arrays
- Pointers
- User Input / Output

Step-by-Step Approach

1. Accept the number of rows and columns.
2. Allocate memory for row pointers using `malloc()`.
3. Allocate memory for each row dynamically.
4. Accept the array elements from the user.
5. Display the matrix.
6. Free memory allocated for each row.
7. Free the memory allocated for row pointers.

Test Cases

Test Case 1

Input

```
Rows = 2
Columns = 3

1 2 3
4 5 6
```

Output

```
1 2 3
4 5 6
```

Test Case 2

Input

```
Rows = 3
Columns = 3

1 2 3
4 5 6
7 8 9
```

Output

```
1 2 3
4 5 6
7 8 9
```
